

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук
Кафедра автоматизированных систем управления

Лабораторная работа №5

по дисциплине «Машинное обучение»

Студент М-РИТ-25-1 _____ Станиславчук С. М.

(подпись, дата)

Руководитель

Доцент, д.т.н. _____ Сараев П. В.

(подпись, дата)

Липецк 2025

1. Задание кафедры

1) Для множества даных из л.р. 4 для одной из моделей (далее - базовая модель) постройте модели с регуляризацией (L1-регуляризация (LASSO); L2-регуляризация (Ridge); комбинированная регуляризация (ElasticNet)).

2) Настройте гиперпараметры регуляризации.

3) Постройте таблицы для сравнения значений адекватности моделей. Сравните адекватность моделей с базовой моделью

2. Цель работы

Изучить подходы к построению моделей машинного обучнеия с применением методов регуляризации для борьбы с переобучением.

3. Ход работы

В качестве языка программирования для выполнения лабораторной работы, я выбрал Python.

Считаем средние ошибки и R2 используя функции из пакета sklearn:

```
models = {
    "LinearRegression": LinearRegression(),
    "Lasso": Lasso(random_state=42),
    "Ridge": Ridge(random_state=42),
    "ElasticNet": ElasticNet(random_state=42)
}

for target in ["Y1", "Y2"]:
    print(f"\n=== {target} ===")
    for name, model in models.items():
        model.fit(X_train, y_train[target])
        y_pred = model.predict(X_test)
        mse = mean_squared_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)
        print(f"{name:17s} | MSE: {mse:10.3f} | R²: {r2:.3f}")
```

Выбираем лучшие параметры alpha и L1 ratio с шагом равным 10.

```
step = 10
models = {

    "Base": (LinearRegression(), {}),

    "Ridge": (Ridge(), {"model__alpha": np.logspace(-3, 3, step)}),

    "Lasso": (Lasso(max_iter=10000), {"model__alpha": np.logspace(-3, 3,
step)}),

    "ElasticNet": (
        ElasticNet(max_iter=10000),
        {
```

```
            "model__alpha": np.logspace(-3, 3, step),
            "model__l1_ratio": np.linspace(0.1, 0.9, 9)
        },
    ),
}

results = []

target_columns = ['Y1', 'Y2']

for target in target_columns:
    for name, (model, params) in models.items():
        pipe = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
        ])

        grid = GridSearchCV(pipe, params, cv=5, scoring="r2", n_jobs=-1)
        grid.fit(X_train, y_train[target])

        y_pred = grid.predict(X_test)

        mse = mean_squared_error(y_test[target], y_pred)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)

        results.append({
            "Target": target,
            "Model": name,
            "Best Params": grid.best_params_,
            "MSE": mse,
            "RMSE": rmse,
            "MAE": mae,
            "R²": r2
        })
```

Пример выполнения программы:

```
stanik@archlinux::/home/stanik/programmer/++Programmer/5_1/ML/lab/5/src > ./venv/bin/python progr.py
Dataset loaded locally!
     X1     X2     X3      X4   X5  X6   X7  X8
0  0.98  514.5  294.0  110.25  7.0   2  0.0   0
1  0.98  514.5  294.0  110.25  7.0   3  0.0   0
2  0.98  514.5  294.0  110.25  7.0   4  0.0   0
3  0.98  514.5  294.0  110.25  7.0   5  0.0   0
4  0.90  563.5  318.5  122.50  7.0   2  0.0   0
      Y1     Y2
0  15.55  21.33
1  15.55  21.33
2  15.55  21.33
3  15.55  21.33
4  20.84  28.28
(614, 8) (154, 8) (614, 2) (154, 2)

=== Y1 ===
LinearRegression  | MSE:      9.153 | R²: 0.912
Lasso             | MSE:     21.465 | R²: 0.794
Ridge             | MSE:      9.653 | R²: 0.907
ElasticNet        | MSE:     19.978 | R²: 0.808

=== Y2 ===
LinearRegression  | MSE:      9.893 | R²: 0.893
Lasso             | MSE:     21.107 | R²: 0.772
Ridge             | MSE:     10.345 | R²: 0.888
ElasticNet        | MSE:     19.569 | R²: 0.789


Target      Model                                                Best Params      MSE     RMSE      MAE       R²
    Y1       Base                                                        {} 9.153188 3.025424 2.182047 0.912184
    Y1      Ridge                                    {'model__alpha': 0.1} 9.158899 3.026367 2.182872 0.912129
    Y1      Lasso                                  {'model__alpha': 0.001} 9.158164 3.026246 2.183154 0.912136
    Y1 ElasticNet {'model__alpha': 0.001, 'model__l1_ratio': 0.4} 9.175993 3.029190 2.184964 0.911965
    Y2       Base                                                        {} 9.893428 3.145382 2.195295 0.893226
    Y2      Ridge                                    {'model__alpha': 0.1} 9.895537 3.145717 2.194880 0.893203
    Y2      Lasso                                  {'model__alpha': 0.001} 9.894230 3.145510 2.194523 0.893217
    Y2 ElasticNet {'model__alpha': 0.001, 'model__l1_ratio': 0.8} 9.897994 3.146108 2.194583 0.893176
```

3. Вывод

Линейная регрессия показывает наилучшую адекватность для обеих Y. Остальные модели либо совпадают по метрикам, либо чуть хуже. По результатам, регуляризация не улучшает качество, т. к. разница в R2 минимальна. Это говорит о том, что датасет не испытывает проблем с шумами, переобучением и обычной линейной регрессии достаточно для текущего небольшого датасета (150 записей).

Параметры регуляризации были отобраны по функциям из numpy: для параметра alpha — logspace (от -3 до 3, 10 шагов); для параметра l1_ratio — linspace (0.1, 0.9, 9) исходя из результатов R2.

Среди отработанных методов, лучший результат по R2 у линейной регрессии; Lasso метод на втором месте, Ridge на третьем и ElasticNet отработал хуже всех остальных методов.

Приложение 1

```
# Изучить подходы к построению моделей машинного обучнеия с применением методов
# регуляризации для борьбы с переобучением.

# https://www.geeksforgeeks.org/machine-learning/lasso-vs-ridge-vs-elastic-net-ml/

# 1. Для множества даных из л.р. 4 для одной из моделей (далее - базовая модель)
# постройте модели с регуляризацией (L1-регуляризация (LASSO); L2-регуляризация
# (Ridge); комбинированная регуляризация (ElastiocNet)).

# **Старье**

# 1.1 Загрузите из UCI ML Repository базу данных Energy efficiency.
# http://archive.ics.uci.edu/ml/datasets/Energy+efficiency
# В этой базе представлены результаты исследования энергоэффективности
# зданий в виде отопительной (heating) и холодильной (cooling) нагрузки
# в зависимости от параметров

import pandas as pd
import numpy as np

df = pd.read_csv("energy_efficiency.csv")

X = df.iloc[:, :-2]  # first 8 columns (features)
y = df.iloc[:, -2:]  # last 2 columns (targets)

print("Dataset loaded locally!")
print(X.head())
print(y.head())


from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)


# **Конец старья**

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression, Lasso, Ridge, ElasticNet
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

models = {
    "LinearRegression": LinearRegression(),
    "Lasso": Lasso(random_state=42),
    "Ridge": Ridge(random_state=42),
    "ElasticNet": ElasticNet(random_state=42)
```

```
}

for target in ["Y1", "Y2"]:
    print(f"\n=== {target} ===")
    for name, model in models.items():
        model.fit(X_train, y_train[target])
        y_pred = model.predict(X_test)
        mse = mean_squared_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)
        print(f"{name:17s} | MSE: {mse:10.3f} | R²: {r2:.3f}")

# 2. Настройте гиперпараметры регуляризации.
print("\n")
step = 10
models = {

    "Base": (LinearRegression(), {}),

    "Ridge": (Ridge(), {"model__alpha": np.logspace(-3, 3, step)}),

    "Lasso": (Lasso(max_iter=10000), {"model__alpha": np.logspace(-3, 3, step)}),

    "ElasticNet": (
        ElasticNet(max_iter=10000),
        {
            "model__alpha": np.logspace(-3, 3, step),
            "model__l1_ratio": np.linspace(0.1, 0.9, 9)
        },
    ),
}


# 3. Постройте таблицы для сравнения значений адекватности моделей. Сравните
# адекватеонсть моделй с базоваой моделью
results = []

target_columns = ['Y1', 'Y2']

for target in target_columns:
    for name, (model, params) in models.items():
        pipe = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
        ])

        grid = GridSearchCV(pipe, params, cv=5, scoring="r2", n_jobs=-1)
        grid.fit(X_train, y_train[target])

        y_pred = grid.predict(X_test)

        mse = mean_squared_error(y_test[target], y_pred)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(y_test[target], y_pred)
        r2 = r2_score(y_test[target], y_pred)

        results.append({
            "Target": target,
            "Model": name,
            "Best Params": grid.best_params_,
            "MSE": mse,
            "RMSE": rmse,
            "MAE": mae,
            "R²": r2
        })

df_results = pd.DataFrame(results)
print(df_results.to_string(index=False))
```